

DSPy

A Beginner-Friendly Tutorial

"Program, don't prompt, your language models."

What's Inside

This tutorial takes you from zero to productive with DSPy. You'll learn WHY DSPy was created, understand its core building blocks — Signatures, Modules, and Optimizers — and see each concept come to life with runnable Python examples. No prior LLM framework experience required.

6 Chapters

Python 3.10+

~30 min read

MIT License

Table of Contents

1. The Problem with Raw Prompting

- Why prompt strings are fragile
- Reproducibility & maintenance challenges
- Enter DSPy: a better abstraction

2. What is DSPy?

- Origin: Stanford NLP
- Core philosophy
- When to use DSPy

3. Installation & Setup

- pip install
- Configure your LM
- Quick smoke test

4. Signatures — Declaring Your Task

- Inline string signatures
- Class-based signatures
- Field types & descriptions

5. Modules — Executing Your Task

- dspy.Predict
- dspy.ChainOfThought
- dspy.ReAct (with tools)
- Building custom modules

6. Optimizers — Auto-Improving Your Program

- What optimizers do
- BootstrapFewShot walkthrough
- Saving & loading optimized programs
- Putting it all together: RAG example

Chapter 1 The Problem with Raw Prompting

Why hand-writing prompt strings leads to pain

If you've worked with LLMs before, you've probably done something like this:

```
# The "classic" approach to using an LLM
import openai

def answer_question(question, context):
    prompt = f"""You are a helpful assistant.
    Use the following context to answer the question.
    Context: {context}
    Question: {question}
    Answer: """

    response = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

# Works... until it doesn't.
answer = answer_question("What is the capital of France?", "France is a country...")
```

This works for toy examples. The problems start when you need to:

- Change the model (GPT-4 → Claude → Llama) — you rewrite prompt formatting logic everywhere.
- Improve quality — you tweak the prompt string by hand, guess-and-check, and have no systematic way to measure improvement.
- Add a new output field — you grep through your codebase hoping you parse it consistently.
- Reproduce results — a different API version or slight wording change silently changes behaviour.
- Scale to multi-step pipelines — passing raw strings between steps becomes a spaghetti mess.



The root cause: you're working at the wrong level of abstraction. You're managing prompt strings instead of describing tasks.

What we actually want

We want to describe WHAT we want (inputs → outputs), not HOW to word the prompt. We want the framework to handle prompt construction, output parsing, caching, and optimization. That's exactly what DSPy provides.

Chapter 2 What is DSPy?

A declarative framework for programming language models

DSPy (Declarative Self-improving Python) is an open-source framework from Stanford NLP (first released December 2022, now at 35k+ GitHub stars and used in production at Shopify, Dropbox, AWS, and others). Its core insight is:



Instead of writing prompt strings, you declare typed input/output signatures. DSPy compiles those into optimised prompts automatically.

The three pillars of DSPy

Concept	What it means
Signature	Declares the task: what goes in and what comes out (like a function signature).
Module	Executes the task: wraps a Signature with a strategy (predict, reason, use tools...).
Optimizer	Improves the program: automatically tunes prompts and few-shot examples against a metric.

DSPy vs. hand-written prompts

	Hand-written Prompts	DSPy
Prompt ownership	You write every word	DSPy generates & optimizes
Model swap	Rewrite prompt logic	Change one line (<code>dspy.LM(...)</code>)
Output parsing	Custom string parsing	Automatic, type-checked
Improving quality	Manual iteration	Run an Optimizer with your metric
Multi-step pipelines	String concatenation	Compose Modules like functions
Reproducibility	Fragile	Cacheable, saveable, loadable

Chapter 3 Installation & Setup

Get DSPy running in under two minutes

Install DSPy

```
pip install -U dspy
```

DSPy requires Python 3.10 or later. It pulls in LiteLLM under the hood, so it can talk to virtually every LLM provider: OpenAI, Anthropic, Cohere, local Ollama models, and more.

Configure a language model

Tell DSPy which model to use by creating an LM object and calling `dspy.configure`:

```
import dspy

# OpenAI
lm = dspy.LM("openai/gpt-4o-mini")    # set OPENAI_API_KEY in env

# Anthropic Claude
lm = dspy.LM("anthropic/claude-3-haiku-20240307") # ANTHROPIC_API_KEY

# Local model via Ollama (free, runs on your machine)
lm = dspy.LM("ollama/llama3", api_base="http://localhost:11434")

# Register it globally
dspy.configure(lm=lm)
```

* Tip: store your API key in an environment variable, never hard-code it. Use `python-dotenv` or `export OPENAI_API_KEY="sk-..."` in your shell.

Smoke test

Verify everything works with the simplest possible DSPy program:

```
import dspy

lm = dspy.LM("openai/gpt-4o-mini")
dspy.configure(lm=lm)

# A one-liner program: topic -> fun_fact
get_fact = dspy.Predict("topic -> fun_fact")
result = get_fact(topic="the Moon")

print(result.fun_fact)
# => "The Moon moves about 3.8 cm further from Earth every year."

# Inspect what prompt was actually sent
dspy.inspect_history(n=1)
```

Chapter 4 Signatures

The declarative heart of every DSPy program

A Signature is the contract for your AI task. It says: "given these inputs, produce these outputs." Think of it like a typed function signature — except the implementation is the language model.

1. Inline string signatures (quick start)

```
# Format: "input1, input2 -> output1, output2"

# Single input, single output
summarize = dspy.Predict("document -> summary")

# Multiple inputs and outputs
translate = dspy.Predict("text, source_lang, target_lang -> translation")

# Call just like a function
result = translate(
    text="Hello, world!",
    source_lang="English",
    target_lang="Spanish"
)
print(result.translation)  # => "Hola, mundo!"
```

The field names you choose matter — DSPy uses them to guide the model. "summary" produces a summary; "bullet_points" would produce bullets instead.

2. Class-based signatures (recommended for production)

For richer control — descriptions, type hints, and docstrings — define your signature as a Python class:

```

import dspy
from typing import Literal

class SentimentAnalysis(dspy.Signature):
    """Classify the sentiment of a product review."""

    review: str = dspy.InputField(
        desc="The product review text to classify"
    )
    sentiment: Literal["positive", "negative", "neutral"] = dspy.OutputField(
        desc="The overall sentiment of the review"
    )
    confidence: float = dspy.OutputField(
        desc="Confidence score between 0.0 and 1.0"
    )
    reasoning: str = dspy.OutputField(
        desc="Brief explanation of the classification"
    )

# Use it with any Module
classify = dspy.Predict(SentimentAnalysis)
result = classify(review="This laptop is incredibly fast and the battery lasts all day!")

print(result.sentiment)    # => "positive"
print(result.confidence)   # => 0.97
print(result.reasoning)    # => "Review highlights speed and battery life positively."

```

- * The docstring becomes the task instruction in the prompt. Write it clearly — it directly affects model behaviour. More descriptive = better outputs.

3. What types can output fields be?

Concept	What it means
<code>str</code>	Free-form text (default)
<code>int / float</code>	Numbers — DSPy parses and validates automatically
<code>bool</code>	True/False decisions
<code>list[str]</code>	List of strings (e.g. extracted keywords, steps)
<code>Literal[...]</code>	Constrained choice from a fixed set (great for classification)
<code>list[dict]</code>	Structured records (e.g. extracted entities)
<code>dspy.Image</code>	Images for multimodal tasks

Chapter 5 Modules

Choosing HOW your signature gets executed

A Module wraps a Signature with a strategy. All modules share the same interface — you call them like functions — but they differ in HOW they get the LM to produce the answer.

dspy.Predict — direct completion

The simplest module. Sends the signature to the LM and returns the output. Use when the task is straightforward and doesn't need step-by-step thinking.

```
import dspy

class ExtractKeywords(dspy.Signature):
    """Extract the main keywords from a piece of text."""
    text: str = dspy.InputField()
    keywords: list[str] = dspy.OutputField(desc="Top 5 keywords")

extract = dspy.Predict(ExtractKeywords)
result = extract(text="Machine learning models learn patterns from data...")
print(result.keywords)
# => ["machine learning", "models", "patterns", "data", "training"]
```

dspy.ChainOfThought — reason before answering

Adds a hidden "reasoning" step before producing the final output. The model thinks step-by-step, which dramatically improves accuracy on complex questions. Just swap Predict for ChainOfThought — no other code changes.

```
import dspy

class SolveMathProblem(dspy.Signature):
    """Solve the math problem step by step."""
    problem: str = dspy.InputField()
    answer: float = dspy.OutputField()

# With dspy.Predict the model might just guess.
# With ChainOfThought it works through the problem first.
solver = dspy.ChainOfThought(SolveMathProblem)
result = solver(problem="A train travels 120 km in 1.5 hours. What is its speed in km/h?")

print(result.reasoning)  # => "Speed = distance / time = 120 / 1.5 = 80"
print(result.answer)     # => 80.0
```

dspy.ReAct — reason AND use tools

ReAct (Reasoning + Acting) lets your program take actions — searching the web, running calculations, querying databases — and incorporate the results into its thinking. You define tools as plain Python

functions.

```
import dspy

# Define tools as regular Python functions
def search_web(query: str) -> str:
    """Search the web and return a relevant snippet."""
    # In real code: call a search API (e.g., DuckDuckGo, Bing)
    results = my_search_api(query)
    return results[0]["snippet"]

def calculate(expression: str) -> float:
    """Evaluate a mathematical expression safely."""
    return eval(expression) # Use a safe evaluator in production!

class ResearchQuestion(dspy.Signature):
    """Answer a question by searching for information and reasoning."""
    question: str = dspy.InputField()
    answer: str = dspy.OutputField(desc="Detailed, sourced answer")

# Pass tools as a list
agent = dspy.ReAct(ResearchQuestion, tools=[search_web, calculate])
result = agent(question="What is the GDP per capita of France?")
print(result.answer)
```

Building custom modules

For multi-step pipelines, subclass `dspy.Module` and define your logic in `forward()`. Each sub-task stays independently inspectable and optimizable.

```
import dspy

class RAG(dspy.Module):
    """Retrieval-Augmented Generation: find context then answer."""

    def __init__(self, retriever):
        # Each step is its own named module
        self.retrieve = retriever
        self.generate = dspy.ChainOfThought(
            "question, context -> answer"
        )

    def forward(self, question: str):
        # Step 1: retrieve relevant passages
        passages = self.retrieve(question, k=3)
        context = " ".join(passages)

        # Step 2: generate an answer grounded in context
        result = self.generate(question=question, context=context)
        return result

# Use it just like any other DSPy module
rag = RAG(retriever=my_vector_db)
answer = rag(question="What are transformer attention heads?")
print(answer.answer)
```

Chapter 6 Optimizers

Automatically improve your program with data

Optimizers are where DSPy truly shines. Instead of hand-tuning prompts, you give DSPy a metric (a function that scores outputs) and a training set (example inputs/outputs), and it automatically finds better prompts and few-shot examples.



Real-world result: Shopify reported a ~550x cost reduction by using DSPy optimizers to compress a frontier-model pipeline into a smaller, optimized model.

How optimization works

The optimizer loop works like this: it generates candidate programs (different prompt instructions and few-shot examples), evaluates each candidate against your metric on the training set, and keeps the best performer. It's automated prompt engineering.

Step 1 — Define a metric

```
def accuracy_metric(example, prediction, trace=None):
    """Return 1 if the prediction matches the gold answer, else 0."""
    return int(prediction.answer.strip().lower() == example.answer.strip().lower())
```

Step 2 — Prepare a training set

```
# dspy.Example wraps your input/output pairs
trainset = [
    dspy.Example(
        question="What is the capital of France?",
        answer="Paris"
    ).with_inputs("question"),
    dspy.Example(
        question="Who wrote Romeo and Juliet?",
        answer="William Shakespeare"
    ).with_inputs("question"),
    # ... ideally 20-200 examples for good results
]
```

Step 3 — Compile with BootstrapFewShot

```

import dspy
from dspy.teleprompt import BootstrapFewShot

# Define your program
class QA(dspy.Module):
    def __init__(self):
        self.answer = dspy.ChainOfThought("question -> answer")
    def forward(self, question):
        return self.answer(question=question)

program = QA()

# Before optimization
result = program(question="What is photosynthesis?")
print("Before:", result.answer)

# Run the optimizer
optimizer = BootstrapFewShot(metric=accuracy_metric, max_bootstrapped_demos=4)
optimized_program = optimizer.compile(program, trainset=trainset)

# After optimization — same API, better answers!
result = optimized_program(question="What is photosynthesis?")
print("After:", result.answer)

# Save for later (no need to re-run the optimizer)
optimized_program.save("qa_optimized.json")

# Load it back
loaded = QA()
loaded.load("qa_optimized.json")

```

Available optimizers at a glance

Concept	What it means
BootstrapFewShot	Auto-selects few-shot examples from training data. Great starting point.
BootstrapFewShotWithRS	Same but uses random search across many candidate sets. More thorough.
MIPROv2	Optimizes both instructions AND examples. Best for structured tasks.
GEPA	Reflective prompt evolution via LLM feedback. Newer, very powerful.
BootstrapFinetune	Goes further: fine-tunes the model weights, not just the prompt.

Key Takeaways

1. Signatures declare your task as typed inputs/outputs — no prompt strings. 2. Modules execute signatures with different strategies (Predict, ChainOfThought, ReAct, custom). 3. Optimizers automatically improve programs against your metric — no manual prompt tuning. 4. Swapping models is one line: `dspy.configure(lm=dspy.LM("..."))`. 5. Programs are composable, cacheable, saveable, and loadable.

Where to go next

- Official docs & tutorials: dspy.ai

- GitHub (35k stars): github.com/stanfordnlp/dspy
- Discord community: discord.gg/XCGy2WDCQB
- Try the RAG tutorial: dspy.ai/tutorials/rag/
- Explore MIPROv2 for serious optimisation: dspy.ai/api/optimizers/MIPROv2/